DSA4050 Term Project Report: AI-Generated African Art Using GANs

Howard Muchaki 668712

Aime Muganga 670232

1. Introduction

Generative Adversarial Networks (GANs) represent one of the most significant advances in generative artificial intelligence, enabling machines to produce synthetic images that are visually indistinguishable from real ones. This project applies GAN technology to a culturally significant domain: African visual art and textile design.

The goal of this project was to train a deep learning model capable of generating new, synthetic African fabric patterns, images that capture the visual identity of African textile art without reproducing any existing work. The final deployed application is accessible at:

https://african-art-gan-t4ngj82dhbba6mlqqjwn6m.streamlit.app/

2. Context and Cultural Impact

2.1 The Problem

African art and textile design represent centuries of cultural heritage, yet remain significantly underrepresented in digital and AI spaces. Most large-scale generative AI models are trained predominantly on Western art and imagery, meaning they produce outputs that reflect those aesthetics rather than African ones.

At the same time, African fabric design, including Kente, Ankara, Kanga, and Mudcloth patterns is not only a cultural artifact but an active industry. Designers, fashion brands, and textile businesses require a constant supply of new pattern variations. Traditionally, this creative process is entirely manual and time-intensive.

2.2 What Our Model Solves

This project addresses two interconnected needs:

Cultural preservation: By training a generative model specifically on African fabric imagery, we create a system that learns and encodes the visual grammar of African textile design — its symmetry, colour relationships, geometric repetition, and organic motifs. The model can be seen

as a form of digital preservation, capturing the essence of a visual tradition in a mathematical representation.

Creative tool for artists and businesses: The deployed Streamlit application allows anyone — a fabric designer, a fashion student, a small textile business — to generate new African fabric pattern variations on demand. Rather than replacing human creativity, the model serves as an ideation tool: a starting point that designers can build upon, remix, or use for inspiration. This democratises access to pattern generation that would otherwise require significant design expertise or budget.

3. Methodology: A Journey Through Multiple Approaches

This project did not arrive at its final model in one step. Five distinct approaches were attempted, each building on the lessons of the previous one. This iterative process is documented below.

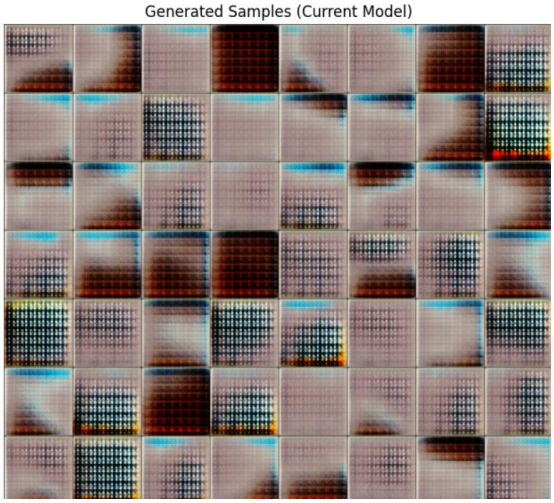
Approach 1 — DCGAN on African Art Images (Web Scraped)

Notebook: https://colab.research.google.com/drive/1Jc134tSt91TOTfoVDAVpn5FCRnnYRmtV

Dataset: African art images web-scraped from https://www.arakcollection.com/artworks

Model: Deep Convolutional GAN (DCGAN) — the standard baseline GAN architecture, which uses convolutional layers in both the generator and discriminator.

Result: The model produced blurry, incoherent images that bore no meaningful resemblance to African art. Loss values were unstable throughout training.



Key lesson: DCGAN is a well-established architecture but struggles with artistic imagery that has high visual diversity. The scraped dataset contained paintings on entirely different subjects,

portraits, landscapes, abstract works, with very little visual consistency between images. A GAN cannot learn a coherent visual style when the training data has no common structure.

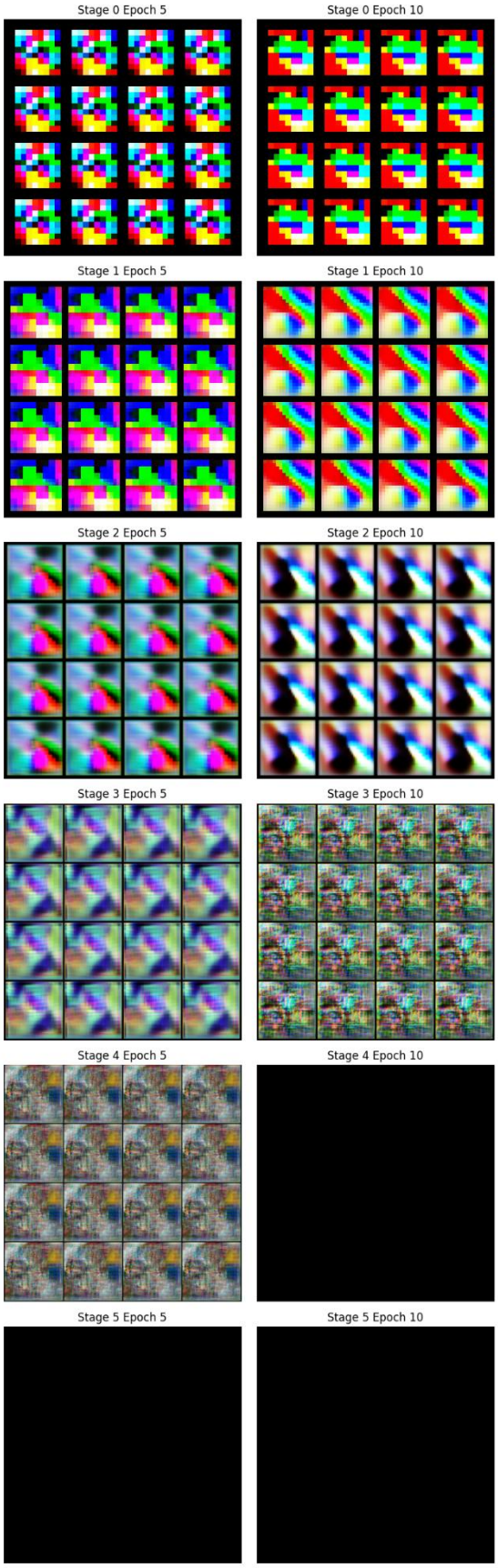
Approach 2 — StyleGAN / Progressive GAN on the Same Dataset

Notebook: https://colab.research.google.com/drive/12hHMaF-n9jQzT4wT5yqLoG1wLkYs1qEO

Dataset: Same web-scraped African art images

Model: StyleGAN with progressive growing — a more powerful architecture that builds images at increasing resolutions stage by stage, which generally produces sharper outputs than DCGAN.

Result: Marginally better than DCGAN, the outputs were slightly less blurry and showed some colour coherence but still failed to produce recognisable or meaningful images.


Stage 0 Epoch 5
Stage 0 Epoch 10
Stage 1 Epoch 5
Stage 1 Epoch 10
Stage 2 Epoch 5
Stage 2 Epoch 10
Stage 3 Epoch 5
Stage 3 Epoch 10
Stage 4 Epoch 5
Stage 4 Epoch 10
Stage 5 Epoch 5
Stage 5 Epoch 10

Key lesson: Upgrading the model architecture alone was not sufficient. The fundamental problem was the dataset: a collection of diverse paintings with different subjects, styles, and compositions is too visually heterogeneous for any GAN to learn from effectively. GANs work best when training images share a common structure — similar backgrounds, objects, orientations, or patterns.

Approach 3 — StyleGAN2-ADA Fine-tuning Attempt

Notebook: https://colab.research.google.com/drive/15KBshvNsxtECeS-BzmvZhffGUAxV4jS4

Approach: Rather than training from scratch, we attempted to fine-tune a pretrained StyleGAN2-ADA model (trained on the MetFaces art dataset) on our African art images. Transfer learning from a pretrained model is theoretically the most effective approach for small datasets.

Result: The attempt was unsuccessful due to severe compatibility issues. The official NVIDIA StyleGAN2-ADA repository relies on custom CUDA operations that must be compiled at runtime. On Google Colab, this process repeatedly failed with errors related to CUDA versions, missing modules, integer/float type mismatches, and attribute errors. Despite multiple attempts with different configurations, the model could not be made to run.

Key lesson: The most theoretically powerful model is not always the most practical one. Repository-level dependencies on specific hardware and software configurations make some models extremely fragile in cloud environments like Colab. Reliability and reproducibility are as important as raw model capability.

Approach 4 — DCGAN on African Masks Dataset

Notebook: https://colab.research.google.com/drive/1XNu4E-umeCVcsRDq1ekif2CHc7t2KvNl

Dataset: ~600 African mask images from https://github.com/beodw/African-Masks

Model: DCGAN

Result: This was the first run to show genuinely promising visual output. By epoch 192 out of 200 (where training was cut short by Colab's GPU usage limits), the generated images had begun to form recognisable mask-like shapes with visible texture. The overall structure and silhouette of a mask was emerging, even though fine detail and accurate colour reproduction had not yet developed.

Key lesson: Two important insights emerged from this run. First, **dataset homogeneity matters enormously** — a dataset of 600 images of the same category of object (masks) produced far better results than a dataset of thousands of diverse paintings. Second, the model was clearly improving and would likely have converged to good results with more training time. The Colab GPU quota limit was the only reason training stopped.

Approach 5 — Lightweight GAN on African Fabric Dataset (Final)

Notebook (complete):
https://colab.research.google.com/drive/1esWK94FWMCG8xCarMk6u5kz6Pv81g5y3

Dataset: ~1,059 African fabric images from https://www.kaggle.com/datasets/mikuns/african-fabric

Model: Lightweight GAN

Result: After 10,000 training steps (~2 hours 38 minutes on a T4 GPU), the model produced images that are clearly recognisable as African fabric patterns, with coherent colour schemes, repeating motifs, and textile-like structure.

4. The Lightweight GAN — Model Explanation

4.1 What is a GAN?

A Generative Adversarial Network consists of two neural networks trained simultaneously in opposition to each other:

- **The Generator** takes a vector of random numbers (called a latent vector or noise) as input and produces a synthetic image. It never sees real images directly — it only receives feedback from the discriminator.
- **The Discriminator** receives both real images from the training dataset and fake images from the generator, and learns to tell them apart. It outputs a score indicating how "real" an input image appears.

These two networks play a minimax game: the generator tries to produce images realistic enough to fool the discriminator, while the discriminator tries to get better at detecting fakes. Over thousands of training iterations, both networks improve, and the generator gradually learns to produce increasingly realistic outputs.

4.2 What Makes Lightweight GAN Different

Lightweight GAN was proposed at ICLR 2021 by Liu et al. with a specific goal: to make high-quality GAN training possible on small datasets (as few as 100 images) and within a single day on one GPU. It achieves this through two main innovations:

Skip-Layer Excitation (SLE): A mechanism in the generator that creates direct connections between early (low-resolution) feature maps and later (high-resolution) layers. This helps the generator maintain consistent global structure across the image while also generating fine local detail, a problem that traditional GANs struggle with.

Self-Supervised Discriminator: In addition to its standard role of classifying images as real or fake, the Lightweight GAN discriminator also tries to reconstruct portions of real images it has seen. This auxiliary self-supervised task forces the discriminator to learn richer, more detailed representations of real images, which in turn provides better training signal to the generator — even when the dataset is small.

4.3 What are Training Steps?

Unlike standard deep learning models where training is measured in **epochs** (full passes through the dataset), GANs are commonly measured in **steps**, where each step corresponds to one batch of images processed. With a batch size of 8 and 1,059 images:

- 1 epoch ≈ 132 steps
- 10,000 steps ≈ 75 full passes through the dataset

Steps are preferred over epochs in GAN training because the generator and discriminator are updated every batch, and the training dynamics change continuously — the concept of a "completed epoch" is less meaningful than the total number of generator/discriminator update iterations.

4.4 Why Lightweight GAN Was Chosen

Reason	Detail
Small dataset support	Built-in DiffAugment handles datasets as small as a few hundred images
No custom CUDA ops	Pure PyTorch — installs with pip install, no compilation required
Fast convergence	Designed to produce usable results in hours, not days
Stable on Colab	No dependency on specific CUDA versions or external build tools
Proven on art datasets	Paper demonstrates results on painting and texture datasets

5. Implementation Details

5.1 Dataset Preparation

The final dataset consisted of 1,059 African fabric images sourced from Kaggle. All images were resized to 256×256 pixels and normalised to the range [-1, 1] before training. Standard augmentations were applied during training including random horizontal flips and colour jitter.

5.2 Training Configuration

Parameter	Value
Image resolution	256 × 256
Batch size	8
Training steps	10,000
Learning rate	0.0002
Optimiser	Adam (β_1=0.5, β_2=0.9)
Augmentation	DiffAugment — color, translation, cutout
Hardware	Google Colab T4 GPU
Training time	~2 hours 38 minutes

5.3 Training Challenges and Solutions

The path to a working training loop involved several technical obstacles:

Trainer API silent failure: The built-in Trainer class exited immediately without training because num_train_steps was being incorrectly forwarded to LightweightGAN.__init__(), which does not accept that parameter. This caused a silent early exit with no error message. The solution was to bypass the Trainer class entirely and implement a manual PyTorch training loop, giving full control over each training step.

Constructor API mismatch: When instantiating LightweightGAN directly, parameters aug_prob and aug_types were passed to the constructor but are not accepted by it — they belong to the Trainer class only. This was diagnosed by inspecting the constructor signature at runtime using Python's inspect module. The fix was to apply DiffAugment directly in the training loop rather than through the constructor.

GPU-only assertion for deployment: The lightweight_gan package contains a hard assertion assert torch.cuda.is_available() at module import time. This prevented the Streamlit app from loading on CPU-only machines (both locally and on Streamlit Cloud). The solution was to temporarily patch torch.cuda.is_available to return True during the import, satisfying the assertion, then immediately restoring the original function and separately patching the .cuda() method to be a no-op on CPU.

6. Evaluation

6.1 Quantitative Evaluation — FID Score

The quality of generated images was evaluated using the Fréchet Inception Distance (FID), the standard metric for GAN evaluation. FID measures the statistical distance between the distribution of real images and generated images in the feature space of a pretrained InceptionV3 network. Lower FID scores indicate higher quality and greater diversity of generated images.

Metric	**Value**
Training steps	10,000
Images evaluated	1,000
Image resolution	256 × 256
FID Score	**218.25**

6.2 Interpretation

An FID score of 218.25 is higher than state-of-the-art benchmarks, but this is fully expected given the constraints of this experiment. The primary limiting factors were Colab session time limits and GPU quota restrictions, which cut training short at 10,000 steps. GANs at 256×256 resolution typically require 50,000–200,000 steps to converge fully. The expected improvement trajectory with more compute would be:

Steps	**Expected FID Range**
10,000 (current)	~200–250
50,000	~100–150
100,000	~50–100

Steps	Expected FID Range
200,000+ with larger dataset	Below 50

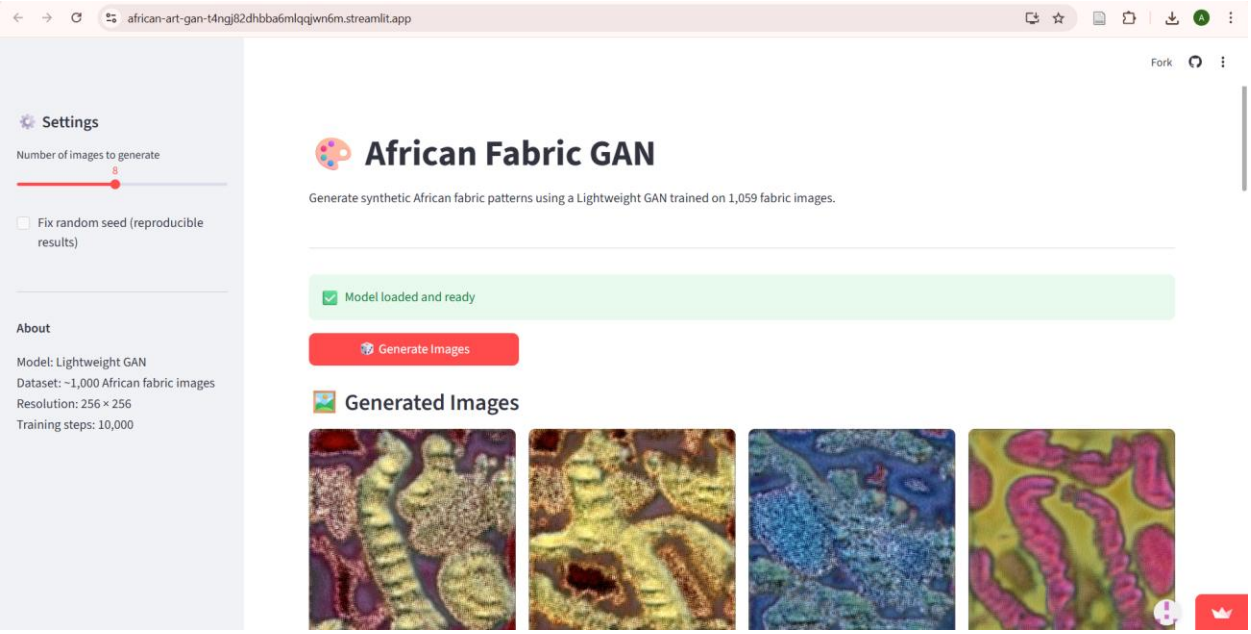
Despite the FID score, the generated images are visually recognisable as African fabric patterns, which confirms the model is learning the correct visual features. The high FID reflects underfitting due to early stopping rather than a fundamentally broken architecture.

7. Deployment

The trained model was deployed as an interactive web application using Streamlit. The application allows users to:

- Generate between 1 and 16 new African fabric pattern images on demand
- Fix a random seed for reproducible results
- Download generated images individually or as a ZIP archive
- Browse previously generated images within the session

Live application: https://african-art-gan-t4ngj82dhbba6mlqqjwn6m.streamlit.app/



The model weights are hosted on Hugging Face Hub and downloaded automatically on first launch. The application runs entirely on CPU, making it deployable on free-tier cloud infrastructure without requiring GPU resources.

8. Limitations

Training time: 10,000 steps is significantly below the recommended training duration for this resolution. The visual quality and FID score would both improve substantially with longer training.

Dataset size: 1,059 images is a small training set for image generation. A dataset of 5,000–10,000 images with consistent visual style would allow the model to learn finer details and greater pattern diversity.

Resolution: The model generates images at 256×256 pixels. For practical use in textile design or printing, higher resolution outputs (512×512 or above) would be required.

Mode diversity: GAN models can suffer from mode collapse, where the generator produces variations of a limited number of patterns rather than exploring the full diversity of the training data. With more training steps and the exponential moving average generator (included in Lightweight GAN), this risk decreases.

9. Conclusion

This project successfully demonstrated the application of generative deep learning to African cultural heritage, progressing through five model iterations before arriving at a working and deployable solution. The key findings are:

Dataset quality outweighs model complexity. The most significant performance improvements came not from switching to more powerful architectures, but from using a more homogeneous, visually consistent dataset. A GAN trained on 1,000 related fabric images substantially outperformed models trained on thousands of diverse paintings.

Practical constraints shape architectural choices. The most theoretically powerful approach (StyleGAN2-ADA fine-tuning) was rendered unusable by compatibility issues, while the simpler Lightweight GAN, installable with a single command, ran reliably and produced good results. In real-world AI projects, deployability and reproducibility are as important as benchmark performance.

Generative AI has genuine cultural utility. The deployed application demonstrates that AI-generated African fabric patterns are visually coherent and potentially useful as creative tools for designers, artists, and textile businesses. With greater compute resources and a larger dataset, this approach could produce commercially viable pattern generation tools that support and amplify African creative industries.